Chapter 6: Advanced Binary Tree Structures

Technical / interview questions, exercises, multiple choice questions and answer key for Chapter 6 (Advanced Binary Tree Structures) of Data Structures.

Technical / Interview Questions

1. Define Advanced Tree Structures.
2. Differentiate between basic BST and self-balancing BST.
3. Why does tree height affect performance?
4. Compare worst-case and amortized complexity with examples.
5. What factors influence the selection of a tree structure?
6. Explain how balancing improves search efficiency.
7. Why are rotations required in balanced trees?
8. What is implicit vs explicit balancing?
9. When is strict balancing preferred?
10. When is relaxed balancing preferred?
11. Define a Threaded Binary Tree.
12. What is meant by a thread in a tree?
13. Why are NULL pointers replaced in threaded trees?
14. Explain single-threaded vs double-threaded trees.
15. What are left-threaded and right-threaded trees?
16. How does inorder traversal work without recursion?
17. What flags are used in threaded representation?
18. Explain insertion in a threaded tree.
19. Why is deletion complex in threaded trees?
20. List advantages of threaded trees.
21. List disadvantages of threaded trees.
22. Mention practical applications of threaded trees.
23. Who introduced AVL trees?
24. Define balance factor.
25. What is the acceptable balance factor range?
26. Define LL rotation.
27. Define RR rotation.
28. Define LR rotation.
29. Define RL rotation.
30. Explain AVL insertion process.
31. Why is AVL deletion more complex?
32. What is maximum number of rotations in AVL insertion?
33. Compare AVL and Red-Black height guarantees.
34. Where are AVL trees preferred?
35. State the properties of Red-Black trees.
36. What is black-height?
37. Why must the root be black?
38. What causes red-red violation?

39. How is insertion fixed in Red-Black trees?
40. Why are Red-Black trees widely used in system libraries?
41. Compare AVL and Red-Black insertion cost.
42. Define splaying.
43. Explain Zig rotation.
44. Explain Zig-Zig rotation.
45. Explain Zig-Zag rotation.
46. Why do Splay trees not guarantee strict height?
47. Define amortized analysis.
48. When are Splay trees most efficient?
49. What is worst-case search time in Splay tree?
50. Compare Splay and AVL trees.

Programming Exercises

1. Write a program to perform inorder traversal of a threaded binary tree without recursion.
2. Convert a BST into an inorder threaded tree.
3. Implement insertion in a double-threaded tree.
4. Implement AVL tree creation.
5. Write function to compute balance factor.
6. Implement AVL insertion with rotations.
7. Implement AVL deletion.
8. Implement Red-Black tree insertion with fix-up.
9. Write a function to calculate black-height.
10. Implement Splay tree insertion.
11. Write program for Zig-Zig and Zig-Zag rotations.
12. Compare runtime of AVL and Splay for skewed input.
13. Write a program to perform inorder traversal of a threaded binary tree without recursion.
14. Convert a BST into an inorder threaded binary tree.
15. Implement insertion in a double-threaded binary tree.
16. Implement deletion in a threaded binary tree.
17. Implement preorder traversal in a threaded binary tree without recursion.
18. Implement postorder traversal using threads.
19. Count number of threads in a threaded tree.
20. Detect whether a given tree is threaded or not.
21. Convert a threaded tree back into a normal BST.
22. Implement search operation in a threaded binary tree.
23. Print predecessor and successor using threads.
24. Implement reverse inorder traversal using threading.
25. Measure memory savings achieved by threading.
26. Implement AVL tree creation from scratch.
27. Write function to compute balance factor of each node.
28. Implement LL rotation.
29. Implement RR rotation.

30. Implement LR rotation.
31. Implement RL rotation.
32. Implement AVL insertion with automatic balancing.
33. Implement AVL deletion with rebalancing.
34. Count number of rotations during AVL insertion.
35. Count number of rotations during AVL deletion.
36. Print AVL tree level-wise.
37. Check whether a given tree satisfies AVL property.
38. Convert sorted array into AVL tree.
39. Measure height of AVL tree after random insertions.
40. Compare height of AVL and normal BST for same input.
41. Implement iterative AVL insertion (without recursion).
42. Implement AVL tree using parent pointers.
43. Write program to detect imbalance point.
44. Generate AVL tree for sequential input and analyze rotations.
45. Implement Red-Black tree insertion with fix-up procedure.
46. Implement Red-Black deletion with fix-up procedure.
47. Implement left rotation in Red-Black tree.
48. Implement right rotation in Red-Black tree.
49. Validate Red-Black tree properties.
50. Count black-height of each node.
51. Detect red-red violation in tree.
52. Verify equal black-height on all paths.
53. Convert AVL tree into Red-Black tree.
54. Compare number of rotations in AVL vs Red-Black insertion.
55. Print Red-Black tree with color labels.
56. Implement iterative Red-Black insertion.
57. Measure average height of Red-Black tree after random input.
58. Implement Red-Black tree using sentinel NIL nodes.
59. Implement search in Red-Black tree.
60. Compare runtime of Red-Black and AVL under heavy updates.
61. Implement Splay tree insertion.
62. Implement Splay tree search with splaying.
63. Implement Splay tree deletion.
64. Implement Zig rotation.
65. Implement Zig-Zig rotation.
66. Implement Zig-Zag rotation.
67. Count number of rotations during splaying.
68. Implement top-down splay algorithm.
69. Implement bottom-up splay algorithm.
70. Measure amortized cost experimentally.
71. Compare runtime for skewed input vs random input.
72. Implement move-to-root heuristic and compare with splay.

73. Compare Splay tree and AVL tree for repeated access.
74. Implement join operation in Splay tree.
75. Implement split operation in Splay tree.
76. Build cache simulator using Splay tree.
77. Analyze worst-case input for Splay tree.
78. Implement menu-driven program to compare BST, AVL, Red-Black, and Splay.
79. Create benchmark program inserting 1 million nodes.
80. Measure search time for 10,000 random queries.
81. Compare memory usage of all four trees.
82. Visualize tree rotations step-by-step in console.
83. Implement file-based storage of AVL tree.
84. Implement persistent AVL tree (functional approach).
85. Implement range query in AVL tree.
86. Implement range query in Red-Black tree.
87. Implement order-statistics tree using Red-Black augmentation.
88. Implement interval tree using Red-Black tree.
89. Implement event scheduler using Splay tree.
90. Simulate OS process scheduler using Red-Black tree.
91. Implement dictionary application using AVL tree.
92. Implement expression tree balancing using AVL rotations.
93. Convert binary heap into AVL tree.
94. Generate worst-case skewed BST and rebalance to AVL.
95. Compare cache locality performance of each tree.
96. Implement threaded AVL tree.
97. Implement colored AVL hybrid tree.
98. Implement hybrid B-Tree + Red-Black approach.
99. Implement lazy deletion in AVL tree.
100. Implement iterative Splay tree without recursion.
101. Simulate LRU cache using Splay tree.
102. Compare theoretical vs empirical complexity of all trees.

Multiple Choice Questions

1. Threaded Binary Trees mainly improve:

(a) Search speed
(b) Traversal efficiency
(c) Height balancing
(d) Deletion

2. In inorder threading, right NULL pointer stores:

(a) Parent
(b) Root
(c) Successor
(d) Height

3. AVL balance factor range is:

(a) -2 to +2
(b) -1 to +1
(c) 0 to 2
(d) 1 to 2

4. Maximum rotations during AVL insertion:

(a) 1
(b) 2
(c) 3
(d) O(n)

5. Height of AVL tree is:

(a) O(n)
(b) O(log n)
(c) O(n log n)
(d) O(1)

6. Red-Black tree guarantees height:

(a) O(n)
(b) O(log n)
(c) $O(n^2)$
(d) O(1)

7. Root of Red-Black tree must be:

(a) Red
(b) Black
(c) Either
(d) Blue

8. Splay tree guarantees:

(a) Worst-case O(log n)
(b) Amortized O(log n)
(c) O(1)
(d) $O(n^2)$

9. Zig-Zig rotation occurs when:

(a) Nodes are opposite sides
(b) Both are left children or both right
(c) Root is red
(d) Height is 2

10. Thread flags indicate:

(a) Height
(b) Color
(c) Thread or child link
(d) Depth

11. Worst-case search in Splay tree:

(a) O(1)
(b) O(log n)
(c) O(n)
(d) O(n log n)

12. Double-threaded tree has:

(a) No threads
(b) Left threads only
(c) Right threads only
(d) Both left and right threads

13. AVL trees are best for:

(a) Update-heavy workloads
(b) Search-heavy workloads
(c) No balancing
(d) Linear storage

14. Red-Black tree insertion requires at most:

(a) 1 rotation
(b) 2 rotations
(c) 3 rotations
(d) 4 rotations

15. Balance factor is:

(a) Left height – Right height
(b) Right – Left
(c) Sum of children
(d) Depth

16. Red-Black tree is less strict than:

(a) BST
(b) Heap
(c) AVL
(d) Splay

17. Black-height ensures:

(a) Perfect balance
(b) Relaxed balance
(c) No balance
(d) Complete tree

18. Threaded trees eliminate:

(a) Stack usage
(b) Rotations
(c) Deletion
(d) Height

19. AVL deletion may require:

(a) No rotations
(b) One rotation
(c) Multiple rotations
(d) None

20. Splay trees are ideal for:

(a) Static data
(b) Skewed access patterns
(c) Disk storage
(d) Heaps

21. In AVL, imbalance occurs when balance factor becomes:

(a) ±1
(b) ±2
(c) 0
(d) ±3

22. Red-Black nodes can be:

(a) Red or Black
(b) Red only
(c) Black only
(d) Blue

23. In threaded trees, successor is found in:

(a) O(n)
(b) O(1)
(c) O(log n)
(d) $O(n^2)$

24. AVL was introduced in:

(a) 1962
(b) 1980
(c) 1995
(d) 2000

25. Splaying moves accessed node to:

(a) Leaf
(b) Middle
(c) Root
(d) Leftmost

26. Red-Red violation occurs when:

(a) Parent and child both red
(b) Both black
(c) Root black
(d) Height 1

27. Threaded trees improve:

(a) Search
(b) Traversal
(c) Height
(d) Balance

28. Maximum AVL insertion rotations:

(a) 1
(b) 2
(c) 3
(d) 4

29. Red-Black trees are commonly used in:

(a) OS kernels
(b) Stacks
(c) Arrays
(d) Queues

30. Splay tree operations are:

(a) Strict O(log n)
(b) Amortized O(log n)
(c) O(1)
(d) $O(n^2)$

31. AVL height is at most:

(a) 2 log n
(b) log n
(c) n
(d) √n

32. In RL rotation, first rotation is:

(a) Right
(b) Left
(c) Double right
(d) None

33. Red-Black tree black-height property avoids:

(a) Skewed tree
(b) Root
(c) Leaf
(d) Thread

34. Threaded trees store extra:

(a) Height
(b) Color
(c) Thread flag
(d) Depth

35. Worst-case Splay search:

(a) O(log n)
(b) O(1)
(c) O(n)
(d) O(n log n)

36. AVL ensures:

(a) Strict height balance
(b) Color balance
(c) No balance
(d) Heap order

37. In Zig rotation, node is:

(a) Grandchild
(b) Child of root
(c) Leaf
(d) Null

38. Red-Black deletion is generally:

(a) Simple
(b) Complex
(c) Impossible
(d) Linear

39. Threaded trees are not suitable for:

(a) Traversal
(b) Balanced search
(c) Low-memory traversal
(d) Iteration

40. Splay trees are self-adjusting by:

(a) Coloring
(b) Threading
(c) Rotations
(d) Height storage

Answer Key

Q	Ans	Q	Ans	Q	Ans	Q	Ans	Q	Ans
1	B	2	C	3	B	4	B	5	B
6	B	7	B	8	B	9	B	10	C
11	C	12	D	13	B	14	B	15	A
16	C	17	B	18	A	19	C	20	B
21	B	22	A	23	B	24	A	25	C
26	A	27	B	28	B	29	A	30	B
31	A	32	A	33	A	34	C	35	C
36	A	37	B	38	B	39	B	40	C